Foreword

The first edition of *Site Reliability Engineering* was published a decade ago, and at the time we weren't sure how much interest there would be in its content. The SRE Book, as it became known, captured much of our accumulated knowledge, experiences, best practices, and ideas for operating Google-scale distributed systems. However, we were concerned that what we wrote about might be too esoteric or Google-specific to be broadly useful. We thought that perhaps a few thousand people might read our book, and fewer still would put it into practice.

I am pleased to say that we may have *slightly* underestimated the interest in our material. Adoption exceeded our wildest expectations, and it has become one of O'Reilly's most popular books. Engineers and organizations across the industry have incorporated ideas, terminology, and challenges from the SRE Book into their own thinking and practices, often finding their own way of implementing, incorporating, or extending the concepts we covered. In the same period, SRE as a discipline has grown and matured, largely outside of Google itself. Within Google SRE, we are both proud of what we have achieved and happy that we have been successful in sharing it.

With another decade of SRE behind us, we also have several new observations about the profession and industry that are worth sharing and that form the subtext for much of what is in this second edition.

First, we find it notable that the growth of SRE has not come at the expense of other approaches to systems management. The You-Build-It-You-Run-It (YBIYRI) approach continues to be a default for many organizations, including several inside Google, and is often the optimal choice early in a service's life. Many of the technical principles covered in the SRE Book—progressive rollout, circuit breakers, cascading failure protection—are independent of the organizational principles, and services that incorporate these technical principles often find less early utility in also adopting SRE organizational structures. This is a sign of progress!

With this continued heterogeneity of operational models, however, we have also observed cases where services and teams that started as YBIYRI have grown to the point where SRE's organizational design would be more efficient for them, but inertia or even

bias impedes the change. Here, a quantitative benchmark is useful to help teams assess their current state. As of this writing, across Google:

- SRE is less than 4% of total engineering headcount.
- Production work consumes less than 15% of total engineering time.

These figures represent a moment in time, and we expect them to further improve over the coming years as we move more toil work into automation. However, we have found this quantitative benchmark to be compelling, since many software as a service (SaaS) teams spend 30%+ of their total engineering time on production work and toil. Faced with the evidence that a sub-4% team can reduce engineering toil by half, the provocative question "Would you like to free up more than 10% of your headcount for feature development?" is often persuasive. Few engineering leaders will turn down extra engineers, and the roughly 4:1 return in engineering cycles on an investment in SRE is a straightforward way for them to get those extra engineers. SRE's core proposition has thus become even stronger as evidence, technology, and practice have accumulated.

Second, over the last decade we increasingly encountered the limits of reliability by policy, and we have been replacing it with reliability by default. We gradually observed that SRE teams across Google had successfully developed automation and tooling for specific classes of tasks, but with three key challenges:

1. Individual SRE teams developed tools that solved their specific use cases but did not work for all other teams. Thus, instead of a single way to (say) roll out a configuration change globally, there were several distinct solutions, each tailored to a subset of systems: stateless versus stateful, regional versus global, etc. Each system had a large enough group of users that it was self-sustaining, and each tried to gain additional adherents in order to get more developers contributing to its codebase.
2. As our collective understanding of reliability principles continued to evolve, each individual system often did not immediately implement the latest findings. Thus, SRE had an excellent set of *policies* governing reliability—for example, "all rollouts to 99.95%+ reliable systems shall be progressive"—but it was up to many individual teams to translate these policies into their specific tools and correctly implement them, and any team that used a tool or system that hadn't implemented the latest policies would be exposed to increased reliability risk.
3. Finally, since reliability tooling was up to individual SRE teams, there was no need for them to adopt the existing systems. If a new type of service didn't neatly work with one of the existing tools, SRE teams were free to create their own—often missing several of the hard-won lessons encoded in existing systems, and again increasing their services' reliability risk.

The combination of these three issues led to both duplicative engineering work—*N* systems each implementing *K* policies—and a number of avoidable outages. The latter issue was eventually dubbed "Treynor's Lament" after I repeatedly observed during postmortems that "There are no new outages at Google, only new instances of old outages." The solution was simple, though not easy:

1. Standardize all teams on a single system for each class of reliability practice (e.g., configuration change management).
2. *Mandate* that all SRE teams use only that system for automating their changes.

The combination of these two requirements was called Convergence and was a program that spanned much of the past decade. Convergence was combined with an increased extra-SRE development focus on common tooling; together, the two produced what we now call the Google Production Platform (GPP). We call this combination reliability by default, because any new SRE team will adopt GPP, extend it as necessary to work with its specific service, and in so doing inherit all of Google's collective reliability experience.

It is worth noting that the Convergence change was not popular with the SRE teams for which it required changing systems, and thus more short-term work! However, there is no arguing with the result: with an SRE team that has maintained a fixed <5% ratio of total engineering headcount, Google has observed an integer (>2!) step-function improvement in its total outage frequency × scope × duration since implementing Convergence.

Third, we continue to find that the early decisions to hold a consistent hiring and skills bar for SRE and product developers but to keep the SRE organization distinct from engineering are both critical to the team's ability to deliver the earlier-cited 4:1 engineering efficiency gain of an investment in SRE relative to YBIYRI. Part of the benefit comes from SRE teams' ability to maintain focus and priority: while feature needs often change quarter to quarter (and sometimes more frequently), SRE teams can remain focused on and personally invested in optimizing reliability and feature velocity. Another part of the benefit is that SREs regularly move out of the team and into other parts of Google software engineering, bringing with them both the experience and the culture from SRE.

Finally, the first edition of *Site Reliability Engineering* opened with "It is a truth universally acknowledged that systems do not run themselves." Part of what is so exciting about our current historical moment is that machine learning is becoming increasingly capable of doing, or automating, ever more of the manual production toil that SREs still tend to. The opportunities for ML to multiply the effectiveness of each SRE are legion, including AI agents looking through the codebase to find all the places where remote procedure calls do not include exponential backoff and fixing them; taking postmortem

action items and looking for similar patterns or latent booby traps throughout the entire codebase, not just the system being reviewed; performing automated STPA risk analysis for *all* systems, not just the handful we have cycles for now; and so much more. It is a heady time for SRE, and I look forward to our progress over the next decade with both excitement and optimism.

Benjamin Treynor Sloss

Previous chapter

< Site Reliability Engineering, 2nd Edition

Next chapter

Preface >